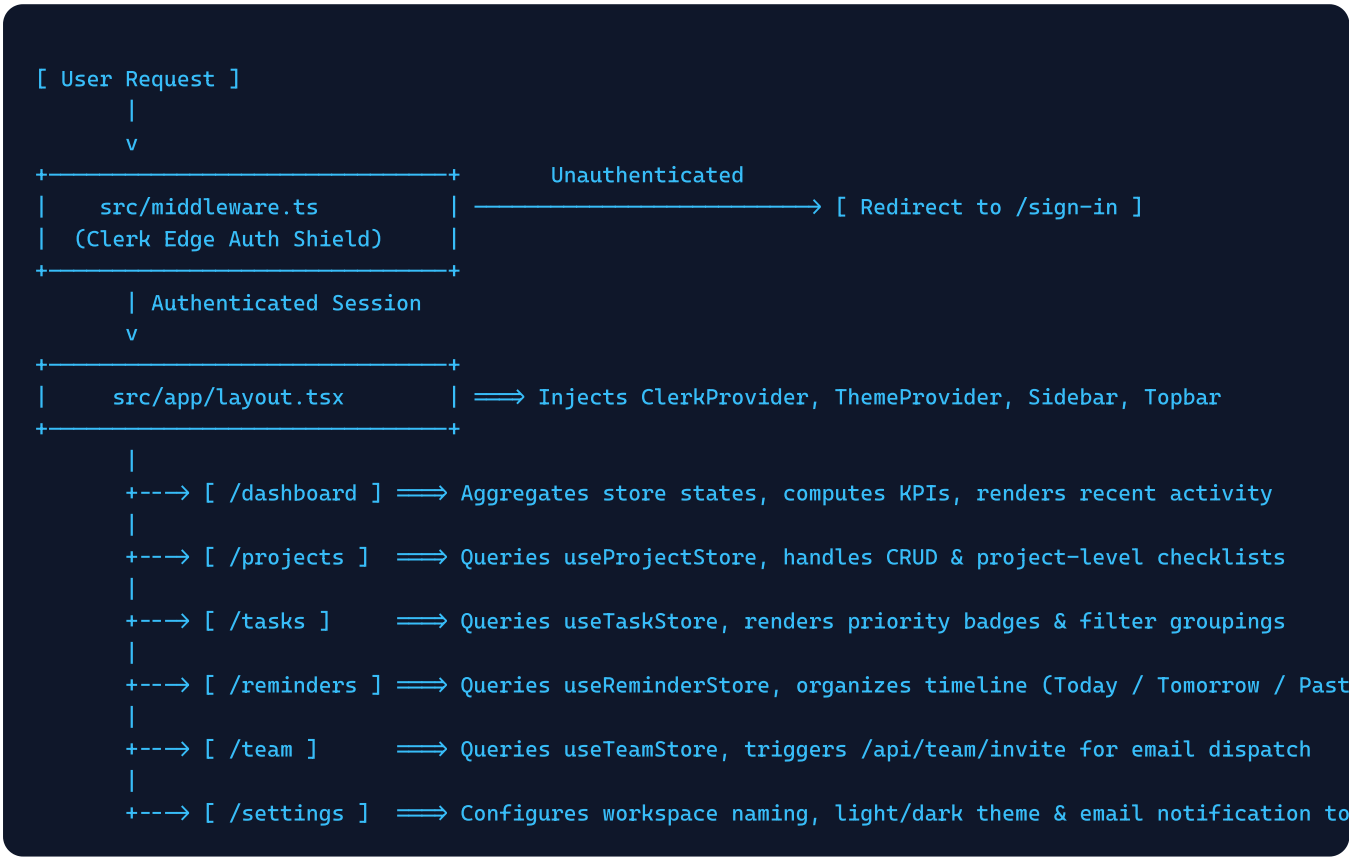
PRONOVA

End-to-End System Workflows & Lifecycle Execution Guide

Document: System Workflow Specifications
Date: 2026-09-04
Engine: Next.js 16 + Zustand + Clerk

1. End-to-End Application Lifecycle

PRONOVA orchestrates interactions across three primary layers: the Edge Authentication Gateway (Clerk), the Reactive Client State Engine (Zustand + LocalStorage), and the Outbound Transactional Services Layer (Nodemailer / Resend).



2. Authentication & Route Protection Workflow

1. **Route Matching & Public Whitelisting**
   `src/middleware.ts` uses `createRouteMatcher(['/sign-in(.*)', '/sign-up(.*)'])`. Any incoming URL request is evaluated against this pattern at the Edge network.

2. **Session Validation & Redirection**
   If the path is private and no valid Clerk session token is detected in cookies, `await auth.protect()` issues an immediate HTTP 307 redirect to the Clerk-hosted or local `/sign-in` page.

3. **Identity Hydration & Topbar Injection**
   Upon successful sign-in, Clerk's `useUser()` hook supplies avatar image, full name, and verified email address to `Topbar`, `HomePage`, and `ProfilePage`.

3. State Persistence & SSR Hydration Guarding

ZUSTAND **Persistent State Flow**

All workspace operations bypass network latency by writing immediately to memory-backed Zustand stores. Zustand's `persist` middleware automatically serializes state into `localStorage`:

- `pronova-projects-storage`: Projects & project subtasks
- `pronova-personal-tasks-storage`: Standalone task list
- `pronova-personal-reminders-storage`: Timeline alerts
- `pronova-team-members-storage`: Active & pending members

REACT 19 **SSR Hydration Safeguard**

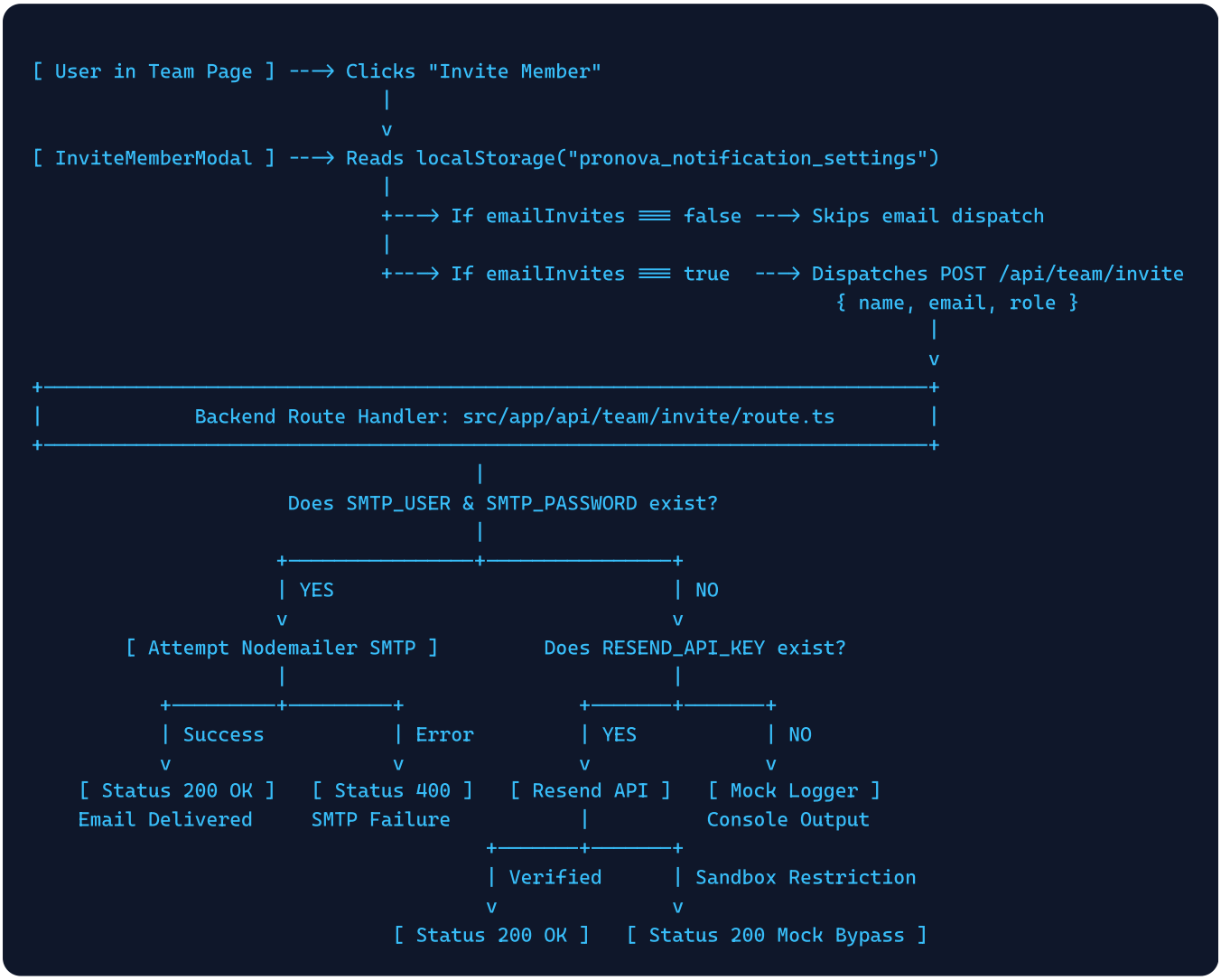
Because Next.js pre-renders HTML on the server where `localStorage` is unavailable, components implement a double-render mount guard:

```
const [mounted, setMounted] = useState(false);
useEffect(() => { setMounted(true); }, []);
if (!mounted) return null; // Or skeleton placeholder
```

This guarantees zero React hydration mismatch warnings across the client and server render trees.

4. Workspace Invitation & Email Delivery Engine Workflow

The most critical server-side operational flow in PRONOVA is the member invitation and transactional email pipeline. The flow executes as follows:



A **Form Input & Smart Domain Generation**

In `InviteMemberModal`, entering a name auto-populates the email using the active admin's corporate domain (e.g. `sarah.miller@example.com`).

B **Origin & Base URL Resolution**

`route.ts` inspects headers `x-forwarded-host` and `x-forwarded-proto` to construct dynamic invitation links matching either `localhost:3000` or the production domain.

C **Delivery Provider Selection**

If Gmail SMTP credentials exist, Nodemailer dispatches an HTML email with workspace branding. If SMTP is unavailable, it queries the Resend transactional REST API.

D **Local Member Creation**

`useTeamStore.inviteMember()` stores the member in browser storage with status `"Pending"` and initials avatar badge, ready for resend or role adjustment.

5. Actionable Roadmap for Resolving Email Sending

To restore 100% reliable email delivery for PRONOVA, the following modifications should be made:

Target Area	Root Cause	Required Technical Fix
.env.local	`SMTP_PASSWORD="mccv vtod rzet acda"` contains spaces from Google's display format.	Remove spaces to supply raw 16-character string: `SMTP_PASSWORD="mccvvtodrzetacda"`.
src/app/api/team/invite/route.ts	Port 465 SSL connection can drop or timeout; code doesn't sanitize password spaces.	Sanitize input with `smtpPassword.replace(/\s+/g, '')`, use `service: 'gmail'` or port 587 STARTTLS.
src/components/team/invite-member-modal.tsx	Modal closes silently even if `response.ok === false`.	Add error state rendering / toast message to notify user when SMTP fails instead of silently closing.